Agentic Research Assistant

A Human-in-the-Loop RAG and Synthesis Framework

Team Members:
Suryansh Chattree
Ishan Maheshwari
Anurag Kumar Tiwari
Sarabjeet Singh

April 2026

Abstract

This project presents an Agentic Research Assistant designed to introduce transparency and granular control into Retrieval-Augmented Generation (RAG) systems. State-of-the-art chatbots and research assistants generally produce high-quality outputs, but they offer users very little ability to tweak the intermediate research steps or audit the source curation. This framework addresses that limitation by utilizing a 13-step pipeline that features dynamic query decomposition, parallel web retrieval, vector embedding aggregation, and a two-tier Large Language Model (LLM) extraction protocol. By incorporating dual-halt user control layers, the system ensures human oversight at critical junctures. The resulting architecture produces policy-grade, heavily cited research briefs with dynamic thematic synthesis, while mitigating API cost bottlenecks through an optimized RAG clustering approach.

1 Problem Statement

Current AI-driven research assistants and chatbots are highly capable and generate genuinely good, coherent outputs. However, their primary limitation lies in their opacity and lack of granular tweakability. Users feed a complex prompt into the system and receive a finished report, but they cannot easily intervene mid-process to adjust search parameters, curate the specific sources being read, or refine the extraction logic.

The objective of this project is to build a system that maintains high-quality synthesis while returning control to the user. Specifically, the system aims to:

- Decompose complex queries into rigorous, mechanism-driven search dimensions.
- Retrieve and process high-density data while discarding conversational fluff.
- Empower the user with maximum control through transparent, halt-driven curation.
- Synthesize findings into cohesive, logically ordered policy briefs.
- Provide a stateless conversational interface for follow-up inquiries.

The challenge lies in engineering a system that supports a strict "Systems Analysis Framework" driven by user feedback, without exceeding computational API limits.

2 Input Description

The system initiates with a single, high-level research query provided by the user.

Rather than relying on a static corpus, the system dynamically generates its dataset via parallel web searches based on the decomposed sub-queries. The user interacts with the data at two critical phases:

- **Query Tuning:** Reviewing and modifying the decomposed search strings before execution.
- **Source Curation:** Manually approving or rejecting scraped URLs based on algorithmic relevance scores, or injecting custom PDFs.

3 System Architecture

The pipeline utilizes a 13-node agentic workflow, separating the retrieval, extraction, and synthesis layers to allow for human intervention.

The system follows a structured pipeline:

1. Query Input
2. Structured Decomposition (Mechanism, Systemic Costs, Differentiation, Policy)
3. **User Control Layer 1** (Query Tuning)

4. Multi-Query Parallel Search
5. Data Cleaning & Text Chunking
6. Vector Embedding Generation
7. Chunk Aggregation & Document Clustering (K-Means)
8. Evaluation Node (Relevance & Density Scoring)
9. **User Control Layer 2** (Source Curation)
10. Chunk Retrieval & Fact Extraction (Fast LLM Boolean Gate)
11. Gatekeeper Node (Zero-Cost Protocol)
12. Final Report Generation (Heavy LLM Synthesis)
13. Stateless Document RAG Chat

4 Methodology

4.1 Structured Decomposition

A Heavy LLM identifies the core entities within the user's prompt and decomposes the query into exactly four highly targeted, keyword-rich search strings. This prevents topic drift and ensures a 360-degree systems analysis.

4.2 Lexical Preprocessing and Embedding

Scraped HTML is stripped of scripts and ads, normalized, and split into paragraphs of 100 to 200 words. These chunks are converted into high-dimensional vector embeddings.

4.3 Aggregation and Clustering

To evaluate sources at the macro level, document-level embeddings are mathematically derived by averaging their constituent chunks:

$$E_{doc} = \frac{1}{N} \sum_{i=1}^{N} E_{chunk_i}$$

K-Means clustering is then applied directly to these aggregated vectors to group sources into thematic clusters, which are presented to the user in the Source Curation dashboard.

4.4 Two-Tier LLM Extraction (The Boolean Gate)

To extract facts, the system retrieves the top relevant chunks for each thematic cluster. A fast, efficient LLM is utilized with a strict "Boolean Gatekeeper" prompt. The model must explicitly return 'true' for both the presence of core entities and concrete data before extracting any facts, effectively pruning irrelevant generalizations.

4.5 Final Synthesis

A Heavy LLM synthesizes the verified, cited facts. The prompt enforces a strict separation of constructs, requires explicit causal chains, and demands dynamic, journalistic section headings rather than rigid sub-query titles.

5 Optimization & Design Decisions

Midway through development, the architecture underwent a major optimization pivot:

- **RAG Implementation:** Initial designs utilized a Heavy LLM to brute-force summarize every retrieved resource in its entirety, which immediately exhausted API free-tier limits. The system was redesigned around Retrieval-Augmented Generation (RAG), embedding chunks and passing only the highest-value data to the models.
- **Stateless Chat Integration:** By shifting to a vector database for the RAG pivot, the system natively unlocked the ability to support a stateless chatbot (Node 13), allowing users to interrogate the research directly without conversational memory drift.

6 Limitations

- **Latency:** The execution of 13 nodes, including web scraping, embedding, clustering, and two human-in-the-loop pauses, results in a multi-minute workflow.
- **Upstream Source Dependency:** The Heavy LLM relies entirely on the quality of the scraped data. If the search API returns low-density sources, the user must actively filter them during the curation halt to prevent a shallow report.

7 Team Contributions

- **Ishan Maheshwari (2401010194):** Defined the overarching problem statement and the transparent system architecture vision. Spearheaded the pitch strategy focusing on maximizing user control and verifiable metrics.
- **Anurag Kumar Tiwari (2401020009):** Designed the core pipeline flow. Managed the multi-query parallel search integration, HTML cleaning, text chunking, and the mathematical aggregation of vector embeddings.
- **Suryansh Chattree (2401010466):** Led major design decisions, including the architectural pivot to a RAG-based system to optimize API costs. Engineered the Boolean Gatekeeper prompt logic and the Heavy LLM policy-brief synthesis framework.
- **Sarabjeet Singh (2401010417):** Conducted limitation analysis regarding latency and source dependency trade-offs. Managed evaluation metrics and laid the groundwork for the stateless RAG chat integration.

8 Conclusion

This project successfully bridges the gap between automated data retrieval and rigorous, user-controlled systems analysis. By combining the semantic reasoning of two-tier LLMs with the geometric clarity of vector clustering and strict human-in-the-loop control, the Agentic Research Assistant produces elite, transparent, and policy-grade research briefs.